BC Basket / Grocery Store Inflation Tracker Handoff Document

1. Project Overview

Explains the project in simple terms.

BC Basket, also known as the Grocery Store Inflation Tracker, is a CISC 4900 project designed to track grocery price inflation using real weekly grocery price data. The project collects prices for a fixed list of grocery items from selected stores, organizes the data in a spreadsheet/database format, calculates monthly averages, and displays the final results on a simple public website. The goal is to make grocery price changes easier for regular shoppers to understand. This project is meant to be continued by future students, not just submitted once. The project was designed for long-term support and meant to be built off of, so future students can keep adding stores, items, and months of data.

---

2. Important Project Links

Important links regarding the project.

Component	Link/Location	Notes
BC-Basket Website	https://bc-cisc4900.github.io/inflation-tracker/	Our current website
GitHub Repository	https://github.com/bc-cisc4900/inflation-tracker	Contains source code, documentation, and other website-related files
Group Management Board	https://github.com/orgs/bc-cisc4900/projects/5	Shows project workflow and task history since the start of the Spring 2026 semester
Group Meeting Notes	https://docs.google.com/document/d/12wIxHZITvYMX94EEUfoCmIPFZYnbcl7tQ3wrw0Mazxk/edit?usp=sharing	All our summarized group meeting notes with our project supervisor, Professor Murray Gross. They outline certain tasks or expectations

		in our project that we would need to meet.
Google Sheet Price Records Workbook	https://docs.google.com/spreadsheets/d/1-U6Gy8KI-ajH7He4ksQ9dPo6TluAoBy75tu9D4PhJR4/edit?gid=881796074#gid=881796074	All 14 weeks of our recorded prices for all 33 items in our grocery items list across 5 different stores, including the 12-Month Breakdown and 14-Week Breakdown.
Supabase Project Backend	https://supabase.com/dashboard/project/jwxxjeovkkmabwyrfgvh	Our backend database dashboard has a very user-friendly interface.
Final Group Presentation Slides		Group presentation outlining all our progress throughout the semester toward this project.
Final Demo Video		Our 10-20 minute recorded group demo demonstrates the entire history of our project, as well as other related components like our group management board or Supabase, and how we record data.

---

3. Team Members and Roles

List of what each group member contributed towards the project.

Member	**Role**	**Main Contributions**
Gabriel Krishtul	Project Manager, Database Manager	Database structure, project coordination, GitHub workflow, data recording standards
Andrew Castillo-Fajardo	Backend Developer	Backend/data-importing scripts, API work, wrote group meet notes, collected

		Trader Joe's data
Mohamad Massoud	Data Collection, QA, Backend/Frontend Support	Website deployment, database/backend debugging, and collected Stop & Shop data
Yuan Ruan	Frontend Developer	Website design/interface, HTML/JavaScript/CSS, and collected Aldi data
Nicholas Cai	Data Collection, QA	Corrected inconsistent data in price entries and noted trends, created the Methods + Credits page on the website, and collected data for Aldi

[Updated Summer 2026] The table above is the Spring 2026 team. For Summer Session 1, Andrew Castillo-Fajardo continued the project alone as Fullstack Developer by completing the database migration, rebuilding the backend API, connecting the website to live data, building the interactive table/comparison features, and maintaining documentation

---

4. What Is Finished Right Now

This section tells the next group what they are inheriting.

Include:

- Final website is working.
- Website is hosted through GitHub Pages.
- Main `index.html` is in the root of the GitHub repository.
- GitHub repo contains source code and documentation.
- 14 weeks of grocery prices were collected.
- FreshDirect was added starting around Week 3, so Weeks 1–2 are blank/missing for FreshDirect.
- Spreadsheet contains weekly sheets plus a final 12-month breakdown.
- Supabase/PostgreSQL replaced the earlier MariaDB direction.
- Final website displays a simplified table instead of filters/sliders.
- Store names are hidden or anonymized in the final public-facing version.

- Final breakdown table is embedded into the website.

Your meeting notes say the group verified the raw data, confirmed Supabase matched the imported Google Sheet data, created 14 weekly Google Sheets plus a final breakdown sheet, embedded the final breakdown table into the website, and updated the website to mention Supabase/PostgreSQL instead of MariaDB.

[Updated Summer 2026] What was added this semester

The Spring version displayed a static, embedded breakdown table. The site is now driven by a live backend:

- **MariaDB → Supabase/PostgreSQL migration completed** — `import.js` and `server.js` fully rewritten for the `pg` driver (`$1` placeholders, `RETURNING`, SSL, `CASE WHEN` averaging).
- **Live Express REST API** (`server.js`, 11 endpoints) serving JSON to the website, with CORS enabled and per-endpoint error logging.
- **Website connected to the live API for the first time** — the pages `index.html`, `method.html`, and `credits.html` were merged into a single scrolling `index.html`.
- **Interactive "Weekly Price Summary" pivot table**: store filter, item search, From/To **month** range filter, **percent-change** indicator per cell, an **Annual Average** column, and **CSV export** (from the filtered data model, not the DOM).
- **"Compare Stores for an Item" view** — powered by the new `/api/prices/compare/:itemId` endpoint, highlighting the cheapest store.
- **Accessibility pass** (ARIA labels, keyboard focus states, table header scopes) and mobile-responsive layout.
- Missing/`0` prices (e.g. FreshDirect Weeks 1–2) are excluded from all averages so they don't skew results.

> Note: the live table only works when the backend (`server.js`) is running. GitHub Pages is static-only, so **the deployed public site currently has no hosted backend** — see Section 11.

---

5. Current Project Structure / Where Everything Is

The GitHub Repository Structure

Folder/File	Purpose

website/	Older/supporting website files
database/	Database schema, SQL, and other database-related files
docs	Documentation, notes, project notes, this document
grocery-importer/	Import/backend scripts or data-processing files
samples/	Recorded data and example outputs
tests/	Test files and expected output checks
README.md	Summarized breakdown of the project
SETUP.md	Set up instructions for new group members

Important: the live GitHub Pages website uses the root-level `index.html`, not necessarily the older files inside the `website/` folder. The advisor asked where the site is hosted, and the group explained it is hosted through GitHub Pages, with the actual `index.html` in the root of the GitHub repository.

[Updated Summer 2026] Key files to know

File	Purpose
`grocery-importer/server.js`	Express REST API (11 endpoints). Run with `node server.js` (port 3000).
`grocery-importer/import.js`	CSV → Supabase import script. `node import.js weekN.csv YYYY-MM-DD [--finalize]`
`grocery-importer/README.md`	Main technical doc: setup, endpoints, verification SQL, and a troubleshooting table. Start here.

`grocery-importer/.env`	DB credentials — **gitignored, you must create it** (see Section 8).
`WebPivot.html`	The entire live website (single merged page) + all frontend JavaScript. The pivot-table and comparison logic is at the bottom of the file. Found in website folder.

The `API_BASE` constant near the bottom of `WebPivot.html` tells the frontend where the API lives (defaults to `http://localhost:3000`).

---

6. Data Collection Rules

This section should explain exactly how future students should collect prices.

Include:

Stores Used

- Aldi
- Key Food
- Stop & Shop
- Trader Joe's
- FreshDirect

Core Rules

- Record prices once per week.
- Track the same item every week.
- Keep brand, package size, item type, and unit consistent.
- Use shelf prices that real consumers see.
- Ignore loyalty-card-only prices.
- Include normal sale/promotional prices if those are visible to all shoppers.
- Record notes if an item is missing, substituted, sold out, or changed size.

- If the exact item is unavailable, use the closest comparable substitute and record the unit price.
- For Aldi or store-brand-heavy stores, use the closest matching store-brand equivalent.

Price collection should be weekly, item definitions should stay consistent, package size matters, substitutions must be documented, and the group should avoid changing the item list too often.

---

7. Spreadsheet / Data Workbook Explanation

The layout of our Google Sheets Workbook where we collect all our grocery price data:

Spreadsheet Tabs

Sheet/Tab	**Purpose**
Recorded prices for weeks 1-14	Raw weekly recorded prices in person
12-Month Breakdown	Final averaged table that's shown as a reverse pivot table on the website
14-Week Breakdown	Used this sheet as a backup copy for the database during migration from MariaDB to Supabase

The weekly sheets contain raw collected prices. The final breakdown sheet is the main output. It averages the prices and organizes the data into a simple table that can be displayed on the website.

The final table structure:

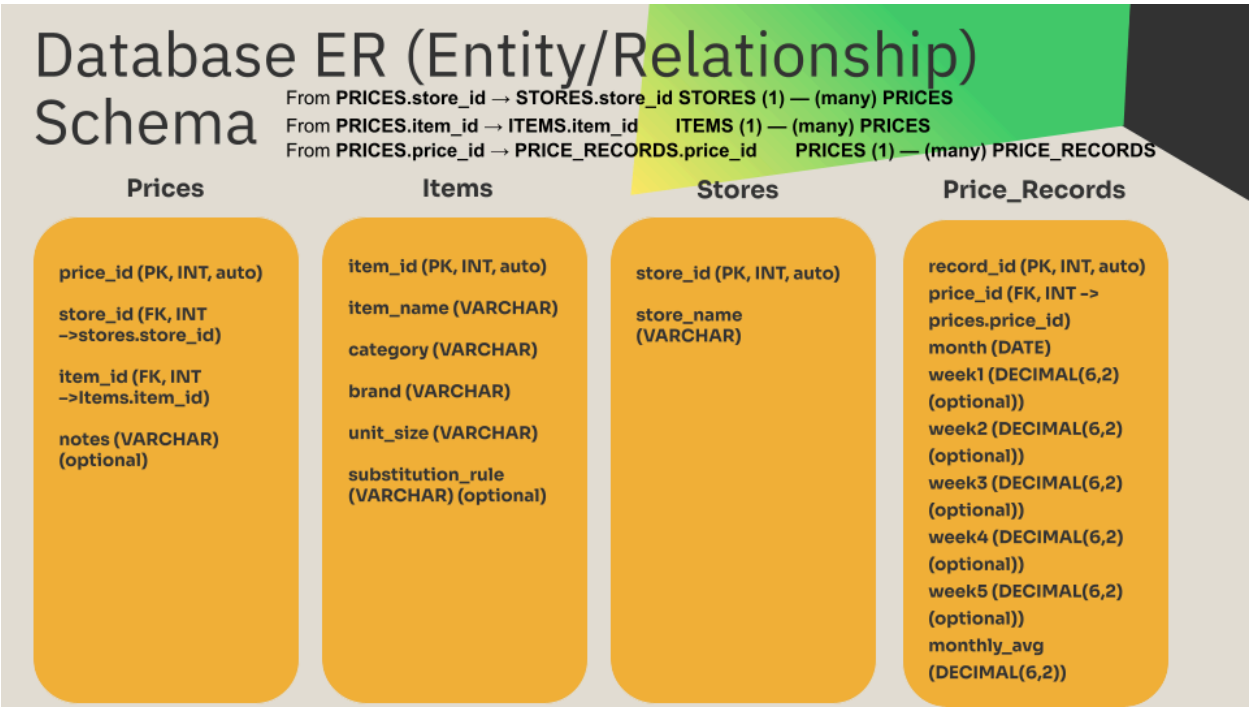
- One row per grocery item.
- Columns represent months.
- Newest month appears first.
- Values are monthly averages.
- Missing months may use `0`, blank, or `-`, depending on the final format.
- Store names should not appear in the final public table.

Our advisor wanted a simple spreadsheet-style table with one row per item, 12 months of data, monthly averages only, no filters, no visuals, no store identifiers, and newest data first.

8. Database / Supabase Explanation

Database Schema:

The project originally used MariaDB, but MariaDB and Brooklyn College UNIX hosting created setup/access problems. The final version moved toward Supabase/PostgreSQL because it is easier for future groups to access and continue. We stepped away from using the Brooklyn College UNIX accounts because the school clears all the data from each account at the end of each semester.



[Updated Summer 2026] ⚠️ Connecting to the database (read this first)

Supabase's direct connection host (`db.<project-ref>.supabase.co`) is IPv6-only. On most home/campus networks (no IPv6), the API starts fine but every query fails with `connect ENETUNREACH 2600:...`. The fix is to use the connection pooler (IPv4) in `grocery-importer/.env`:

---

9. Website Explanation

Explanation of what the website currently does.

- Hosted through GitHub Pages.
- Main file is root `index.html`.
- Displays final simplified table.
- Uses green spreadsheet-style formatting for readability.
- Avoids filters/sliders because the final goal is clarity.
- Public version should avoid showing real store names.
- Credits section includes project/team/repo information.
- Methods section explains how prices and averages are calculated.

Why the website changed:

Earlier versions had filters, sliders, and more interaction. After advisor feedback, the group simplified the site because the target users may be everyday shoppers, older users, or non-technical users. The final version is meant to show the data immediately instead of making users navigate controls. Our advisor accepted the final direction and that the earlier interactive website was “not appropriate” because it assumed too much from the end user. The revised website was meant to be usable by the least technical person possible.

[Updated Summer 2026] What the website does now

The site is now WebPivot`.html` that fetches live data from the API. Sections, top to bottom:

- **Hero / intro** with light/dark toggle.
- **Weekly Price Summary** — the live pivot table: one row per item, one **month** column each (newest first), each cell the average across stores. Includes a store filter, item search, a From/To month range filter, per-cell percent change (red rose / green fell), an Annual Average column, and CSV export.
- **Compare Stores for an Item** — pick an item, see each store's average as a card with the cheapest highlighted.
- **Spreadsheet embed**, **Methodology**, and **Credits** sections.

The Spring team's "keep it simple for non-technical users" goal still holds — the default view is just a readable table, and all filters are optional. The difference is the data is now live from the database instead of a static embed, and the table updates automatically as new weeks are imported. To run it locally: start the API (`node server.js`), serve the site (`python3 -m http.server 8080`), and open `http://localhost:8080/index.html`.

---

10. How to Update the Project Next Semester

Weekly Update Process

1. Choose the week/date.
2. Collect prices from each assigned store.
3. Enter prices into the correct weekly sheet.
4. Mark missing items clearly.
5. Add substitution notes when needed.
6. Check package sizes and units.
7. Update monthly averages at the end of the month.
8. Refresh the 12-Month Breakdown sheet.
9. Confirm the website table still displays correctly.
10. Commit any website/documentation changes to GitHub.

Monthly Update Process

1. Calculate monthly average per item/store.
2. Average across the five stores if using the final public table.
3. Reverse-sort months so newest data appears first.
4. Keep only the latest 12 months visible.
5. Update the website table.
6. Test the live website link.

Our advisor wanted the project to sort the data newest-to-oldest, select the latest 12 entries, calculate averages/percent change, and keep the website focused on the final table.

[Updated Summer 2026] Importing data into the live database

The spreadsheet steps above still apply for collection. To get that data into Supabase (which is what the live website reads), use the import script instead of manual DB edits:

1. Export the week's sheet tab as `weekN.csv`.
2. Run `node import.js weekN.csv YYYY-MM-DD` — use the **same date for every week of a month** so they group into one record.
3. On the **last week of the month**, add `--finalize` to compute and store `monthly_avg`.

4. Verify in Supabase: each month should total **165 records** (33 items × 5 stores). Verification SQL is in `README.md`.
5. The website reflects the new data automatically on reload — no manual table editing needed.

The percent-change and newest-first sorting the advisor asked for are now handled automatically by the frontend, not by hand.

---

11. Known Issues / Things Future Groups Should Fix

Any possible issues we encountered that future groups should look out for:

Include:

- Automating the weekly-to-monthly update process is not fully finished.
- Some FreshDirect data is missing for Weeks 1–2 because it was added later.
- Data collection can be slightly inconsistent if item sizes or brands change.
- Some stores may discontinue items.
- Sales and promotions can affect weekly prices.
- Online prices may not match in-store prices.
- Supabase/database structure may need cleanup or better documentation.
- Website table may need a cleaner update process.
- Future groups should decide whether to keep using placeholders for missing months or replace them with real data over time.
- More testing should be added to verify averages.

[Updated Summer 2026] Current status of these issues + new ones

- **[MOST IMPORTANT] The deployed public site has no hosted backend.** GitHub Pages is static-only, so `server.js` only runs on your own machine. On the live URL the price table shows an error state until someone hosts the backend (Render, Railway, Fly.io, or a Supabase Edge Function). This is the #1 thing to fix.
- **Supabase IPv6 connection** — the direct host fails on networks without IPv6; you must use the connection pooler (Section 8). This is documented but will still trip you up if skipped.
- *Resolved:* automatic monthly-average calculation is now handled by `--finalize` and the API. Percent change, newest-first sorting, and CSV export are also done.
- *Resolved:* the reverse-sort pivot table the Spring team wanted is implemented (newest month first).

- `web2.html` is a deprecated leftover — delete it or ignore it; `index.html` is the real site.
- FreshDirect Weeks 1–2 are still missing; the code now treats those `0` values as "no data" so they don't distort averages.
- The API returns generic errors to the browser; always check the terminal running `node server.js`, which logs the real cause of any failure.

---

12. Future Improvements

Next steps for our project:

- Continue collecting weekly prices.
- Add more months of real data.
- Replace placeholder months with real future data.
- Add more stores only after the current workflow is stable.
- Improve Supabase automation.
- Implement actual reverse sort for the pivot table when there is more months of data
- Create a cleaner admin/data entry form.
- Add automatic monthly average calculation.
- Add CSV export if needed.
- Add simple charts later, but only after the table is reliable.
- Keep the website simple and readable.
- Improve mobile layout.
- Add a clear “How to Continue This Project” section to the README.

The project is intended as a scalable, ongoing inflation tracking system that future group members can continue implementing.

[Updated Summer 2026] Reprioritized next steps

Several Spring "future" items are now done (CSV export, automatic monthly averages, real reverse-sort pivot table, cleaner update process via the import script). The most valuable remaining work, in order:

1. **Host the backend** so the live public site actually serves data — this unlocks everything else and is the single biggest gap.
2. **Add trend-line charts** on top of the existing monthly data (the data shape is already chart-ready).

3. **Keep collecting weekly data** — value compounds; multiple semesters enable real long-term inflation analysis.
4. **Optional: automate Google Sheets → Supabase import** (deliberately kept manual for reliability; keep the manual CSV path as a fallback if you attempt this).
5. **Add more stores/items** — the schema needs no changes; add rows to `stores`/`items` and the dynamic CSV parser handles the rest.
6. Build a simple admin/data-entry form (the API already has import and validation endpoints to support one).

Summer 2026 — Contributor & Quick Start

Continued by: Andrew Castillo-Fajardo (Fullstack Developer, sole developer) — ANDREW.CASTILLOFAJARDO35@bcmail.cuny.edu

Fastest path to a working local setup:

1. `cd grocery-importer && npm install`
2. Create `.env` using the **pooler** host and `postgres.<project-ref>` username (Section 8).
3. `node server.js` (starts the API on port 3000).
4. In another terminal: `cd website && python3 -m http.server 8080`, then open `http://localhost:8080/index.html`.
5. If the table shows an error, read the terminal running `server.js` — it logs the real reason. The most common cause is the IPv6/pooler issue in Section 8.

The complete technical reference (all endpoints, verification queries, and a full troubleshooting table) lives in `grocery-importer/README.md`.